06. 암호학적 검증 및 인증서 체인

암호학적 검증 개요

DCAP Quote 검증에서는 여러 단계의 암호학적 검증이 수행됩니다. ECDSA P-256 서명 검증, X.509 인증서 체인 검증, CRL(Certificate Revocation List) 확인 등이 포함됩니다.

지원하는 암호학적 알고리즘

ECDSA P-256 서명

- **곡선**: NIST P-256 (secp256r1)
- **해시**: SHA-256
- **서명 길이**: 64바이트 (r: 32바이트, s: 32바이트)
- **공개키 길이**: 64바이트 (압축되지 않은 형식에서 0x04 접두사 제외)

지원하는 증명 키 타입

```
pub const ATTESTATION_KEY_TYPE_ECDSA256_WITH_P256_CURVE: u16 = 2;


// Quote 헤더에서 확인
if quote.header.attestation_key_type != ATTESTATION_KEY_TYPE_ECDSA256_WITH_P256
bail!("Unsupported DCAP attestation key type");
}
```

인증서 체인 구조

Intel SGX/TDX 인증서 계층

```
Intel Root CA
↓
Intel Intermediate CA
↓
PCK Certificate (Platform Certification Key)
↓
Quote Signature
```

신뢰 앵커 (Trust Anchor)

```rust
// Intel SGX 루트 CA 인증서 (하드코딩됨)
pub static TRUSTED_ROOT_CA_DER: &[u8] = include_bytes!("TrustedRootCA.der");

pub fn sgx_pck_root() -> TrustAnchor<'static> {
webpki::anchor_from_trusted_cert(&TRUSTED_ROOT_CA)
.expect("Failed to create trust anchor")
}
```

ECDSA 서명 검증 프로세스

1. Quote 서명 검증

```
// ECDSA 공개키 구성 (압축되지 않은 형식)
let mut pub_key = [0x04u8; 65]; // 0x04 접두사 + 64바이트 키
pub_key[1..].copy_from_slice(&auth_data.ecdsa_attestation_key);

// Ring 라이브러리를 사용한 공개키 객체 생성
let peer_public_key = ring::signature::UnparsedPublicKey::new(
&ring::signature::ECDSA_P256_SHA256_FIXED,
pub_key
);

// Quote 서명 검증 (헤더 + 리포트 부분)
peer_public_key
.verify(
raw_quote.get(..signed_quote_len)?, // 서명된 데이터
&auth_data.ecdsa_signature, // 64바이트 서명
)
.map_err(|_| anyhow!("Isv enclave report signature is invalid"))?;
```

2. QE Report 서명 검증

```
// PCK 인증서로 QE Report 서명 검증
```

```
let  asn1_signature = encode_as_der(&auth_data.qe_report_signature)?;

if  qe_leaf_cert

.verify_signature(

webpki::ring::ECDSA_P256_SHA256,

&auth_data.qe_report,

&asn1_signature,

)

.is_err()

{

return Err(anyhow!("Rsa signature is invalid for qe_report in quote"));

}
```

3. TCB 정보 서명 검증

```
// TCB 정보 발급자 인증서로 TCB 정보 서명 검증

let  asn1_signature = encode_as_der(&collateral.tcb_info_signature)?;

if  tcb_leaf_cert

.verify_signature(

webpki::ring::ECDSA_P256_SHA256,

collateral.tcb_info.as_bytes(),

&asn1_signature,

)

.is_err()
```

```
{
return Err(anyhow!("Rsa signature is invalid for tcb_info in quote_collateral"))
}
```

DER 인코딩 및 디코딩

ECDSA 서명 DER 인코딩

```
pub fn encode_as_der(data: &[u8]) -> Result<Vec<u8>> {
if data.len() != 64 {
bail!("Key length is invalid");
}
// ASN.1 SEQUENCE 구성
let mut sequence = der::asn1::SequenceOf::<der::asn1::UintRef, 2>::new();
// r 값 (첫 32바이트)
sequence.add(der::asn1::UintRef::new(&data[0..32])?)?;
// s 값 (다음 32바이트)
sequence.add(der::asn1::UintRef::new(&data[32..])?)?;
// DER 인코딩
let mut asn1 = alloc::vec![0u8; 72]; // 충분한 버퍼 크기
let mut writer = der::SliceWriter::new(&mut asn1);
writer.encode(&sequence)?;
Ok(writer.finish()?.to_vec())
```

```
}
```

DER 인코딩 구조

```
SEQUENCE {
r INTEGER, -- 32바이트 정수
s INTEGER -- 32바이트 정수
}
```

X.509 인증서 체인 검증

메인 인증서 체인 검증 함수

```
pub fn verify_certificate_chain(
leaf_cert: &webpki::EndEntityCert, // 리프 인증서
intermediate_certs: &[CertificateDer], // 중간 인증서들
time: UnixTime, // 검증 시점
crl_der: &[&[u8]], // CRL 목록
) -> Result<()> {
let sig_algs = webpki::ALL_VERIFICATION_ALGS;

// CRL 파싱
let crls: Vec<CertRevocationList> = crl_der
.iter()
```

```
.map(|der| BorrowedCertRevocationList::from_der(der).map(|crl| crl.into()))
.collect::<Result<Vec<_>, _>>()?;
let  crl_slice = crls.iter().collect::<Vec<_>>();


// 폐기 검증 옵션 구성
let  builder = webpki::RevocationOptionsBuilder::new(&crl_slice)?;
let  revocation = builder
.with_depth(webpki::RevocationCheckDepth::Chain) // 전체 체인 검사
.with_status_policy(webpki::UnknownStatusPolicy::Deny) // 알 수 없는 상태 거부
.with_expiration_policy(webpki::ExpirationPolicy::Enforce) // 만료 정책 강제
.build();


// 인증서 체인 검증 수행
leaf_cert
.verify_for_usage(
sig_algs, // 지원하는 서명 알고리즘
&[sgx_pck_root()], // 신뢰 앵커 (Intel Root CA)
intermediate_certs, // 중간 인증서 체인
time, // 검증 시점
webpki::KeyUsage::server_auth(), // 키 사용 목적
Some(revocation), // CRL 검증 옵션
None, // 추가 옵션 없음
)
```

```
.context("Failed to verify certificate chain")?;


Ok(())

}
```

인증서 체인 검증 단계

1. **서명 검증**: 각 인증서의 디지털 서명 확인
2. **체인 연결**: 상위 인증서로 하위 인증서 서명 검증
3. **신뢰 앵커**: 최상위 인증서가 신뢰할 수 있는 루트 CA인지 확인
4. **유효 기간**: 모든 인증서가 검증 시점에 유효한지 확인
5. **CRL 검사**: 인증서가 폐기되지 않았는지 확인

CRL (Certificate Revocation List) 검증

CRL 구성

```
let  crls = [&collateral.root_ca_crl[..], &collateral.pck_crl];
```

루트 CA CRL 검증

```
// 포크된 webpki를 사용하여 루트 CA CRL 검증

dcap_qvl_webpki::check_single_cert_crl(TRUSTED_ROOT_CA_DER, &crls, now)?;
```

CRL 검증 정책

- **RevocationCheckDepth::Chain**: 전체 인증서 체인에 대해 CRL 검사
- **UnknownStatusPolicy::Deny**: CRL에서 상태를 알 수 없는 인증서 거부
- **ExpirationPolicy::Enforce**: CRL 만료 정책 강제 적용

인증서 파싱 및 추출

PEM 형식 인증서 체인 파싱

```
pub fn extract_certs<'a>(cert_chain: &'a [u8]) -> Result<Vec<CertificateDer<'a
let mut certs = Vec::<CertificateDer<'a>>::new();

// PEM 형식 파싱
let raw_certs = extract_raw_certs(cert_chain)?;
for raw_cert in raw_certs.iter() {
let cert = webpki::types::CertificateDer::<'a>::from(raw_cert.to_vec());
certs.push(cert);
}

Ok(certs)
}

pub fn extract_raw_certs(cert_chain: &[u8]) -> Result<Vec<Vec<u8>>> {
Ok(pem::parse_many(cert_chain)?
```

```
.iter()
.map(|i| i.contents().to_vec())
.collect())
}
```

Intel 확장 필드 추출

```
pub fn get_intel_extension(der_encoded: &[u8]) -> Result<Vec<u8>> {
let cert: Certificate = der::Decode::from_der(der_encoded)?;
let mut extension_iter = cert
.tbs_certificate
.extensions
.as_deref()
.unwrap_or(&[])
.iter()
.filter(|e| e.extn_id == oids::SGX_EXTENSION) // OID: 1.2.840.113741.1.13.1
.map(|e| e.extn_value.clone());

let extension = extension_iter.next().context("Intel extension not found")?;
if extension_iter.next().is_some() {
bail!("Intel extension ambiguity"); // 중복 확장 필드 오류
}
Ok(extension.into_bytes())
```

```
}
```

QE Report 해시 검증

QE 해시 데이터 구성

```
// QE 해시 데이터 구성 (96바이트)
let mut qe_hash_data = [0u8; QE_HASH_DATA_BYTE_LEN]; // 96바이트

// ECDSA 증명 키 (64바이트)
qe_hash_data[0..ATTESTATION_KEY_LEN].copy_from_slice(&auth_data.ecdsa_attestatio

// QE 인증 데이터 (32바이트)
qe_hash_data[ATTESTATION_KEY_LEN..].copy_from_slice(&auth_data.qe_auth_data.data
```

SHA-256 해시 계산 및 비교

```
// Ring 라이브러리를 사용한 SHA-256 해시 계산
let qe_hash = ring::digest::digest(&ring::digest::SHA256, &qe_hash_data);

// QE Report의 report_data 첫 32바이트와 비교
if qe_hash.as_ref() != &qe_report.report_data[0..32] {
bail!("QE report hash mismatch");
```

```
}
```

인증서 타입 및 CA 식별

PCK 인증서 타입 확인

```
// 지원하는 인증서 타입: PCK 인증서 체인 (PEM 형식)
if certification_data.cert_type != PCK_CERT_CHAIN { // 5
bail!("Unsupported DCAP PCK cert format");
}
```

CA 타입 식별

```
impl Quote {
pub fn ca(&self) -> Result<&'static str> {
let raw_cert_chain = self.raw_cert_chain()?;
let certs = utils::extract_certs(raw_cert_chain)?;
let cert = certs.first().ok_or(anyhow!("Invalid certificate"))?;
// 인증서 파싱 및 발급자 확인
let cert_der: Certificate = der::Decode::from_der(cert)?;
let issuer = cert_der.tbs_certificate.issuer.to_string();
if issuer.contains(constants::PROCESSOR_ISSUER) { // "Processor"
return Ok(constants::PROCESSOR_ISSUER_ID); // "processor"
```

```
} else if issuer.contains(constants::PLATFORM_ISSUER) { // "Platform"
return Ok(constants::PLATFORM_ISSUER_ID); // "platform"
}
Ok(constants::PROCESSOR_ISSUER_ID) // 기본값
}
}
```

암호학적 상수 및 OID

Intel SGX 확장 OID

```
pub mod oids {
use const_oid::ObjectIdentifier as OID;

pub const SGX_EXTENSION: OID = oid("1.2.840.113741.1.13.1");
pub const PPID: OID = oid("1.2.840.113741.1.13.1.1");
pub const TCB: OID = oid("1.2.840.113741.1.13.1.2");
pub const PCEID: OID = oid("1.2.840.113741.1.13.1.3");
pub const FMSPC: OID = oid("1.2.840.113741.1.13.1.4");
pub const PCESVN: OID = oid("1.2.840.113741.1.13.1.2.17");
pub const CPUSVN: OID = oid("1.2.840.113741.1.13.1.2.18");
}
```

암호학적 상수

```rust
pub const ECDSA_SIGNATURE_BYTE_LEN: usize = 64; // ECDSA 서명 길이
pub const ECDSA_PUBKEY_BYTE_LEN: usize = 64; // ECDSA 공개키 길이
pub const QE_HASH_DATA_BYTE_LEN: usize = 96; // QE 해시 데이터 길이
pub const ATTESTATION_KEY_LEN: usize = 64; // 증명 키 길이
pub const AUTHENTICATION_DATA_LEN: usize = 32; // 인증 데이터 길이
```

에러 처리 및 보안 고려사항

타이밍 공격 방지

- 상수 시간 비교 연산 사용
- Ring 라이브러리의 안전한 암호학적 구현 활용

서명 검증 실패 처리

```rust
.map_err(|_| anyhow!("Isv enclave report signature is invalid"))?;
```

인증서 체인 검증 실패

```rust
.context("Failed to verify certificate chain")?;
```

이러한 다층적 암호학적 검증을 통해 Quote의 무결성과 인증서 체인의 신뢰성이 보장됩니다.